



华南师范大学

本科学生实验（实践）报告

院 系：

实验课程：

实验项目：

指导老师：

开课时间： ~ 年度第 学期

专 业：

班 级：

学生姓名：

华南师范大学教务处

华南师范大学实验报告

学生姓名_____学 号_____

专 业_____年级、班级_____

课程名称_____实验项目_____

实验类型 ☐验证 ☐设计 ☐综合 实验时间_____年____月____日

实验指导老师_____实验评分_____

一、实验内容

(一) 为 Tiny 语言扩充的语法有

- 1.实现改写书写格式的新 if 语句;
- 2.扩充算术表达式的运算符:++(前置自增 1)、--(前置自减 1) 运算符(类似于 C 语言的++和--运算符,但不需要完成后置的自增 1 和自减 1)、求余%、乘方^;
- 3.扩充扩充比较运算符:<(小于)、>(大于)、<=(小于等于)、>=(大于等于)、<>(不等于)等运算符;
- 4 增加正则表达式,其支持的运算符有:或(|)、连接(&)、闭包(#)、括号()、可选运算符(?)和基本正则表达式。
- 5.for 语句的语法规则(类似于 C 语言的 for 语言格式):
书写格式:for(循环变量赋初值;条件;循环变量自增或自减 1) 语句序列
- 6.while 语句的语法规则(类似于 C 语言的 while 语言格式):
书写格式:while(条件) 语句序列 endwhile

(二) 对应的语法规则分别为:

- 1. 把 TINY 语言原有的 if 语句书写格式
`if_stmt-->if exp then stmt-sequence end | if exp then stmt-sequence else
stmt-sequence end`
改写为:
`if_stmt-->if(exp) stmt-sequence else stmt-sequence | if(exp) stmt-sequence`
- 2. ++(前置自增 1)、--(前置自减 1) 运算符、求余%、乘方^等运算符的

语法规则请自行组织。

- 3.<(小于), >(大于), <=(小于等于), >=(大于等于), <>(不等于)等运算符的文法规则请自行组织。
- 4.为 tiny 语言增加一种新的表达式——正则表达式,其支持的运算符有:或(|)、连接(&)、闭包(#)、括号()、可选运算符(?)和基本正则表达式,对应的语法规则请自行组织。
- 5.为 tiny 语言增加一种新的语句, ID==正则表达式 (同时增加正则表达式的赋值运算符==)
- 6.为 tiny 语言增加一个符合上述 for 循环的书写格式的语法规则,
- 7.为 tiny 语言增加一个符合上述 while 循环的书写格式的语法规则,
- 8.为了实现以上的扩充或改写功能,还需要注意对原 tiny 语言的语法规则做一些相应的改造处理。

二、实验目的

- 通过实验掌握如何对现有编程语言进行扩展和改写,以满足特定需求或增加新功能的能力。
- 通过实验理解如何设计新的语法规则和扩展现有语法,考虑语法的一致性、简洁性和易用性。
- 通过实验,在增加新的运算符和语句类型时,培养算法思维和逻辑推理能力。
- 通过实验更深入地理解编程语言的理论基础,包括语法分析、语义分析和编译原理等方面的知识。
- 通过实验将理论知识应用到实际项目中,加深对编程语言和编程范式的理解,并积累实践经验。
- 通过实验在解决扩展和改写过程中遇到各种问题和挑战,提高解决问题的能力 and 调试技巧。

三、实验文档：

1. 实验项目概述

- 实现改写书写格式的新 if 语句；
- 扩充算术表达式的运算符：++ (前置自增 1)、-- (前置自减 1) 运算符 (类似于 C 语言的++和--运算符，但不需要完成后置的自增 1 和自减 1)、求余%、乘方^；
- 扩充扩充比较运算符：<(小于)、>(大于)、<=(小于等于)、>=(大于等于)、<>(不等于)等运算符；
- 增加正则表达式，其支持的运算符有：或(|)、连接(&)、闭包(#)、括号()、可选运算符(?)和基本正则表达式。
- for 语句的语法规则 (类似于 C 语言的 for 语言格式)：书写格式：
for(循环变量赋初值;条件;循环变量自增或自减 1) 语句序列
- while 语句的语法规则 (类似于 C 语言的 while 语言格式)：书写格式：
while(条件) 语句序列 endwhile
- 要提供一个源程序编辑的界面，以让用户输入源程序 (可输入，可保存、可打开源程序)
- 可由用户选择是否生成语法树，并可查看所生成的语法树。
- 应用程序的操作界面应为 Windows 界面。

2. 实验项目设计

- 需求分析：
首先确定了实验项目的需求，扩充 tiny 语言并设计一个应用软件，提供图形化生成语法树的功能。
- 功能设计：
 1. 将代码识别为 token 类型
 2. 根据 token 构建语法树

3. 将语法树用 windows 界面展现出来

4. windows 界面提供的其他功能

3. 数据结构设计

- 原字符识别标识 tokenType: enum 枚举类型

token 又可分为 book-keeping tokens、reserved words、multicharacter tokens、special symbols 四种类型

- 树结构 NodeKind、StmtKind、ExpKind

根据不同的树结构，使用 treeNode 的不同域进行记录

treeNode 定义如下：

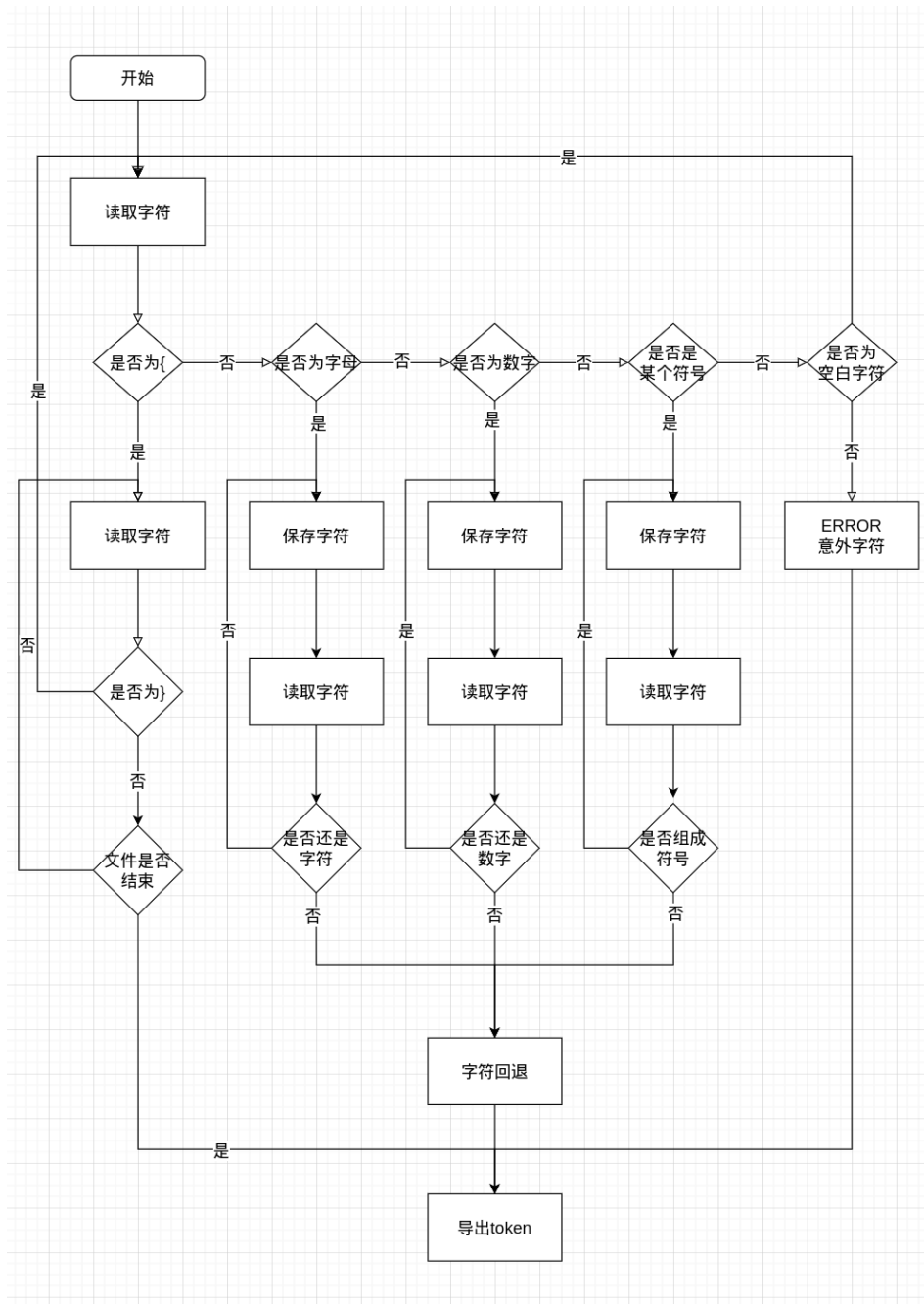
```
typedef struct treeNode
{
    struct treeNode * child[MAXCHILDREN];
    struct treeNode * sibling;
    int lineno;
    NodeKind nodekind;
    union { StmtKind stmt; ExpKind exp;} kind;
    union { TokenType op;
            int val;
            char * reBaseExp;
            char * name; } attr;
    ExpType type; /* for type checking of exps */
} TreeNode;
```

4. 关键算法设计

从 tiny 编译器中可以获取大部分的逻辑，只需要通过修改和新增文法、类型即可得到最终程序

- getToken(): 从原字符中读取字符转为 token

大致流程图如下



- pause(): 读取字符，建树
根据文法写出递归子程序即可
修改后的全部文法如下：

EBNF中TINY语言的文法

```
program → stmt-sequence
stmt-sequence → statemnet { ; statement } [ ; ]
statement → if-stmt | repeat-stmt | assign-stmt | read-stmt | write-stmt | while-stmt | for-stmt
if-stmt → if ( exp ) stmt-sequence [else stmt-sequence] ;
repeat-stmt → repeat stmt-sequence until exp
assign-stmt → normal-assign | re-assign
read-stmt → read identifier
write-stmt → write exp
while-stmt → while ( exp ) stmt-sequence endwhile
for-stmt → for ( assign-stmt ; exp ; exp ) stmt-sequence ;
normal-assign → identifier := exp
re-assign = identifier == re-exp
exp → simple-exp [comparison-op simple-exp]
comparison-op → < | > | <= | >= | =
simple-exp → term [addop term]
addop → + | -
term → power-exp [mul-op power-exp]
mul-op → * | / | %
power-exp → factor [^ factor]
factor → ( exp ) | number | [increase-op] identifier
increase-op → ++ | --
re-exp → simple-re-exp { | simple-re-exp }
simple-re-exp → closure { [ & ] closure }
closure → re-fector [ closure-op ]
closure-op → # | ?
re-fector = re-ele | ( re-exp )
re-ele → number | identifier { number | identifier }
```

其中额外的改动有：

stmt 中可以匹配单独的 ; , 使得 if 和 for 可以以)为开始, 分号为结尾来标识语句块, 效果同 C 语言中用{}包裹语句块

递归子程序书写示例如下：

- statement → if-stmt | repeat-stmt | assign-stmt | read-stmt | write-stmt | while-stmt | for-stmt

```
TreeNode *statement(void)
{
    TreeNode *t = NULL;
    switch (token)
    {
        case IF:
            t = if_stmt();
            break;
        case REPEAT:
```

```

        t = repeat_stmt();
        break;
case ID:
    t = assign_stmt();
    break;
case READ:
    t = read_stmt();
    break;
case WRITE:
    t = write_stmt();
    break;
case WHILE:
    t = while_stmt();
    break;
case FOR:
    t = for_stmt();
    break;
default:
    syntaxError("unexpected token -> ");
    printToken(token, tokenString);
    token = getToken();
    break;
} /* end case */
return t;
}

```

5. 实验项目测试

测试案例的编写：

对每一个修改过的语法都进行测试，这里展示几个比较简单的测试：

正确案例：

{test 4: power-exp}

exp := 12 ^ --exp

{test 5: term}

exp := a * c ^ 15;

exp := a % c;

exp := a / c

错误案例：

{test 1: normal-assign}

exp := {错误：空表达式}

{test 2: factor}

exp := (1; {错误：未闭合的括号}

exp := 1); {错误：未匹配的括号}

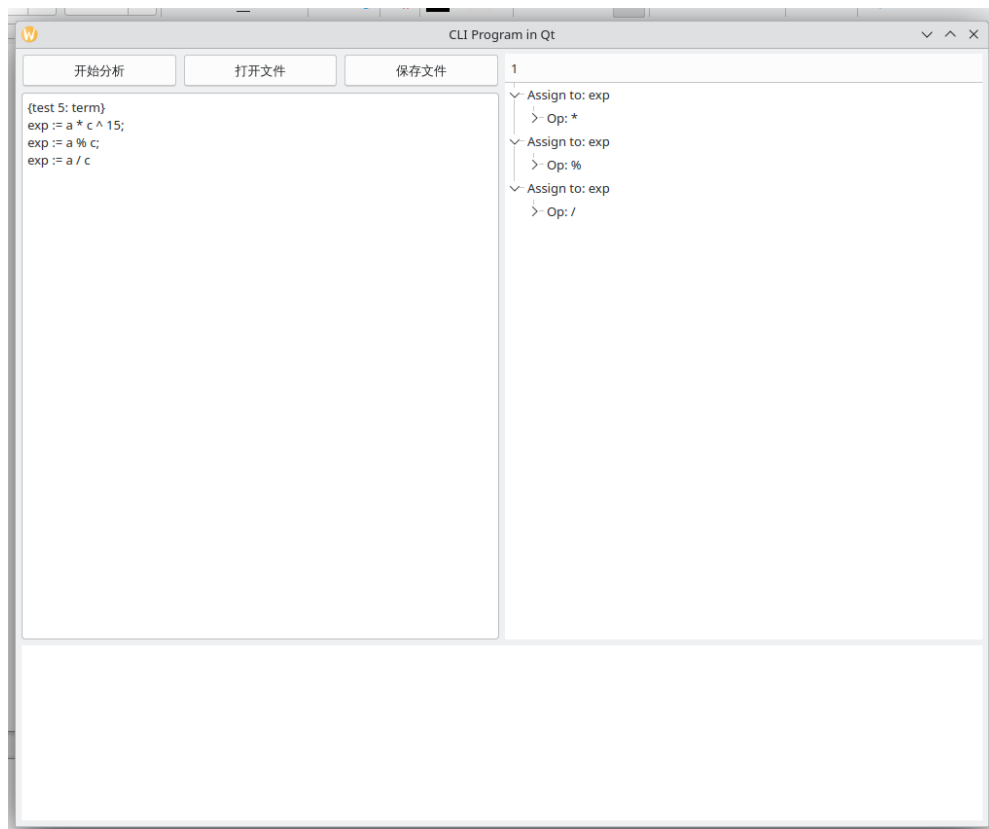
exp := num\$ber; {错误：非法字符 \$}

exp := 123abc; {错误：数字和字母的混合变量名}

exp := 1 + ; {错误：操作符后缺少操作数}

exp := *2 {错误：操作符前缺少操作数}

对案例的测试结果：



- 其他测试结果均通过，这里不再显示
- 完整测试文件位于 testfile 文件夹内，按类型分成小测试

四、实验总结（心得体会）

在完成这次实验之后，我有以下几方面的心得体会：

- 在这次实验中，详细的需求分析是至关重要的。通过明确每一个功能需求，我们能够更好地理解要实现的功能以及如何实现。提前书写好文法，这使得后续的设计和编码工作更加顺畅，也减少了因为需求不明确而导致的反复修改。
- 扩展现有语言的语法需要考虑很多方面的因素。首先，要确保新语法的引入不会破坏原有语法的正确性和一致性。其次，需要设计合理的文法规则，使得新语法能够被正确解析。这次实验增强了我对编程语言语法设计和解析器实现的理解。
- 为 Tiny 语言增加正则表达式支持是一个有挑战性的任务。正则表达式本身是一种强大的工具，增加对其的支持需要考虑各种运算符及其组合的解析。这次实验使我对正则表达式的内部机制有了更深入的了解，同时也认识到在设计语法时需要考虑的复杂性。
- 在引入新运算符（如前置自增、自减、求余和乘方运算符）时，需要确保这些运算符的优先级和结合性与现有运算符一致。这次实验使我认识到在扩展语法时，运算符的优先级和结合性设计是非常关键的，否则会导致解析器无法正确解析表达式。
- 通过为 Tiny 语言增加 for 和 while 循环的支持，我体会到编程语言设计的灵活性和扩展性。不同的语言设计有其独特的语法和结构，通过这次实验，我学会了如何将其他语言的有用特性引入到新的语言中，同时保持语言的一致性和简洁性。
- 在实现新的语法规则后，测试和调试是保证语法正确性和稳定性的重要步骤。这次实验中，通过大量的测试用例，验证了新语法的正确性，并发现和修正了一些隐藏的问题。测试和调试不仅帮助我找出了问题，还加深了我对新语法规则的理解。

五、参考文献：

- [1] Kenneth C.Louden.编译原理及实践[M].机械工业出版社,2000-3-1.
- [2] 陆文周.Qt5 开发及实例（第 2 版）[M].第 2 版.电子工业出版社,2015-5-1.

